

ЛАБОРАТОРНА РОБОТА 4

РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

FileCraft

Посилання на SRS:

[https://docs.google.com/document/d/1_tlhRhy0UAwX0fyLTk8tAGB9ywhCu4dM/edit?usp=drive_link &oid=106294426388174620542&rtpof=true&sd=true](https://docs.google.com/document/d/1_tlhRhy0UAwX0fyLTk8tAGB9ywhCu4dM/edit?usp=drive_link&oid=106294426388174620542&rtpof=true&sd=true)

Таблиця Code Review одnogрупника для мого коду:
fileItem.cs:

yehorbilokon2 opened 4 days ago

Collaborator

...

Рядок коду	Проблема (Code Smell)	Категорія	Рекомендація щодо виправлення
string[] allowed = { ".pdf", ".png", ... }; (Метод Upload)	Допустимі розширення файлів "зашиті" прямо в тіло методу. Якщо їх треба змінити, доведеться лізти в логіку.	Magic Strings / Порушення Open-Closed	Винести масив у статичну константу на рівні класу: private static readonly string[] AllowedExtensions = { ".pdf", ... };
Метод Upload цілком	Метод одночасно: перевіряє дані, валідує розширення, створює звіти. Це занадто багато дій для одного місця.	Порушення SRP / Довгий метод	Застосувати Extract Method: винести логіку перевірки розширення у private bool IsValidExtension(string fileName), а створення звіту — у private void CreateUploadReport(int dataLength).
ShareUrl = \$"https://share.example.com/files/{FileId}/{safeName}";	Базовий домен жорстко закодований. Якщо домен зміниться, код перестане працювати коректно у нових середовищах.	Magic Strings (Маричні рядки)	Винести "https://share.example.com/files/" у const string BaseUrl на рівні класу або, що краще, передавати його через конструктор (Dependency Injection).
Блок foreach (char c in FileName) { ... safeChars.Add("_"); } у TogglePublic	Формування рядка по одному символу через List є неефективним та займає багато місця (10 рядків коду).	Excessive Complexity (Надмірна складність)	Використати регулярні вирази (Regex) або StringBuilder. Наприклад: string safeName = Regex.Replace(FileName.ToLower(), @"[^a-z0-9\-_]", "_"); (це 1 рядок замість 10).
Цикл foreach (Tag existing in Tags) { if (...) return; } у AddTag	Ручний перебір колекції для пошуку дублікатів робить код довшим і менш читабельним.	Code Duplication (Винахід колеса)	Замінити цикл на використання вбудованих методів LINQ: if (Tags.Any(t => t.TagId == tag.TagId)) return;

unitTests:

yehorbiokon2 opened 4 days ago Collaborator ...			
Рядок коду (або блок)	Проблема (Code Smell)	Категорія	Рекомендація щодо виправлення
Усі блоки Arrange:FileItem file = new FileItem { FileId = 1, UserId = 10... };	Створення об'єкта FileItem дублюється у кожному з 17 тестів.	Code Duplication (Дублювання коду)	Винести ініціалізацію базового об'єкта FileItem метод з атрибутом [SetUp], який буде автоматично створювати новий екземпляр перед кожним тестом, або створити приватний метод CreateValidFileItem().
Тести 1, 2, 3 (перевірка null, "", " ")	Три окремі тести перевіряють одну й ту саму логіку валідації порожнього рядка для методу Upload.	Code Duplication / Порушення DRY	Об'єднати ці три тести в один параметризований тест, використовуючи атрибути NUnit: [TestCase(null)], [TestCase("")], [TestCase(" ")].
UserId = 10, UserId = 5, UserId = 7	Використання різних ID користувачів та файлів без явної причини. Це створює ілюзію, що ID якось впливає на результат тесту.	Magic Numbers (Магічні числа)	Винести дефолтні значення у константи на рівні класу, наприклад: private const int ValidUserId = 10;. Використовувати їх скрізь, окрім тестів, де і специфічно перевіряється.
file.Upload("DATA");, file.Upload("PDF DATA");	Жорстко закодовані рядки даних. Якщо формат валідації зміниться, тести впадуть.	Magic Strings (Магічні рядки)	Замінити на константи або змінні зі зрозумілими назвами, наприклад: string invalidFileData = "DATA"; та передавати її в метод.
Тест 11: Assert.IsTrue(url.Contains("my_file_.pdf"));	Очікуваний результат ("my_file_.pdf") жорстко зашитий прямо у виклик Assert. Це приховує логіку перетворення.	Magic Strings (Погане іменування / Магічні рядки)	Створити змінну в блоці Arrange: string expectedSanitizedFileName = "my_file_.pdf"; і використовувати її в Assert.IsTrue(url.Contains(expectedSanitizedFileName));

Посилання на Code Review:

<https://github.com/yuliapanasenko-source/FileCraft/issues/7>

<https://github.com/yuliapanasenko-source/FileCraft/issues/6>

Мої до коду одногрупника:

<https://github.com/yehorbilokon2/bse-lr1-bilokon/issues/6>

<https://github.com/yehorbilokon2/bse-lr1-bilokon/issues/5>

Результати рефакторингу мого коду:

fileItem.cs:

Magic Strings -> константа AllowedExtensions

Було:

// всередині методу Upload:

```
string[] allowed = {  
    ".pdf", ".png", ".jpg",  
    ".jpeg", ".docx", ".txt"  
};  
  
bool validExtension = false;  
foreach (string ext in allowed)  
{  
    if (FileName.EndsWith(ext, ...))  
    { validExtension = true; break; }  
}  
  
if (!validExtension)  
    throw new InvalidOperationException(...);
```

стало:

// на рівні класу FileItem:

```
private static readonly string[] AllowedExtensions =  
    { ".pdf", ".png", ".jpg",  
      ".jpeg", ".docx", ".txt" };
```

// у методі Upload:

```
if (!IsValidExtension(FileName))
```

```
    throw new InvalidOperationException(...);
```

Чому: масив був «захований» у тілі методу — при зміні дозволених розширень треба було шукати логіку. Константа на рівні класу — єдина точка змін; не порушує Open-Closed принцип.

Extract Method -> IsValidExtension + CreateUploadReport

Було:

```
// Upload робить все сам:
```

```
public FileItem Upload(string fileData)
```

```
{
```

```
    // ...валідація...
```

```
    bool validExtension = false;
```

```
    foreach (string ext in allowed)
```

```
    { /* 8 рядків */ }
```

```
    if (!validExtension) throw ...;
```

```
    Reports.Add(new Report {
```

```
        ReportId = Reports.Count + 1,
```

```
        ParentFileId = FileId,
```

```
        CreatedAt = DateTime.UtcNow,
```

```
        Content = $"Файл '{FileName}'..."
```

```
    });
```

```
    return this;
```

```
}
```

Стало:

```
// Upload описує лише «що»:
```

```

public FileItem Upload(string fileData)
{
    // ...валідація...
    if (!IsValidExtension(FileName))
        throw ...;
    CreateUploadReport(fileData.Length);
    return this;
}

```

```

private static bool IsValidExtension(
    string fileName) { ... }

```

```

private void CreateUploadReport(
    int dataLength) { ... }

```

Чому: Upload порушував SRP — одночасно перевіряв дані, валідував розширення, формував звіт. Після Extract Method кожен метод має одну відповідальність.

Ручний foreach по символах -> Regex.Replace

Було:

```

// 10 рядків для санітизації:
var safeChars = new List<char>();
foreach (char c in FileName)
{
    if (char.IsLetterOrDigit(c)
        || c == '.' || c == '-')
        safeChars.Add(char.ToLower(c));
}

```

```

        else
            safeChars.Add('_');
    }
    string safeName =
        new string(safeChars.ToArray());
    стало:
    // 1 рядок:
    string safeName = Regex.Replace(
        FileName.ToLower(),
        @"[^a-z0-9\-.]",
        "_");

```

Чому: ручний автомат по символах — надмірна складність (Excessive Complexity). Regex декларативно виражає той самий намір в одному рядку й легше читається.

Ручний foreach -> LINQ Any() у AddTag

Було:

```

// ручний пошук дублікату:
foreach (Tag existing in Tags)
{
    if (existing.TagId == tag.TagId)
        return;
}

```

Tags.Add(tag);

Стало:

```

// вбудований LINQ:
if (Tags.Any(t => t.TagId == tag.TagId))

```

```
return;
```

```
Tags.Add(tag);
```

Чому: Any() виражає намір («чи існує хоч один?») значно чіткіше за ручний цикл і є стандартним підходом у C#.

unitTests.cs:

Magic Numbers -> константи DefaultFileId / DefaultUserId

Було:

```
// розкидано по 17 тестах:
```

```
FileItem file = new FileItem {  
    FileId = 1,  
    UserId = 10, // або 5, або 7  
    FileName = "document.pdf"  
};
```

Стало:

```
// один раз на рівні класу:
```

```
private const int DefaultFileId = 1;  
private const int DefaultUserId = 10;  
FileItem file = CreateValidFileItem();
```

Чому: різні UserId = 5 / 7 / 10 без пояснення створювали ілюзію, що конкретне значення впливає на результат. Константи документують намір.

Фабричний метод CreateValidFileItem() замість дублювання

Було:

```
// в кожному з 17 тестів:
```

```
FileItem file = new FileItem
```

```
{  
    FileId = 1,  
    UserId = 10,  
    FileName = "document.pdf"  
};
```

Стало:

// один метод — всі тести:

```
private FileItem CreateValidFileItem(  
    string fileName = ValidPdfFileName)  
=> new FileItem {  
    FileId = DefaultFileId,  
    UserId = DefaultUserId,  
    FileName = fileName  
};
```

// у тесті:

```
FileItem file = CreateValidFileItem();
```

Чому: при зміні структури FileItem достатньо оновити один метод, а не редагувати всі 17 тестів. Класичний патерн Object Mother / фабрика.

3 окремих тести -> один параметризований [TestCase]

Було:

// три окремих тести:

[Test]

```
public void Upload_NullData_Throws...()
```

```
{ /* arrange + act + assert */ }
```

[Test]

```
public void Upload_EmptyData_Throws...()
```

```
{ /* дублікат структури */ }
```

[Test]

```
public void Upload_WhitespaceData_Throws...()
```

```
{ /* дублікат структури */ }
```

Стало:

```
// один параметризований тест:
```

```
[TestCase(null, typeof(ArgumentNullException))]
```

```
[TestCase("", typeof(ArgumentException))]
```

```
[TestCase(" ", typeof(ArgumentException))]
```

```
public void Upload_InvalidFileData_Throws(
```

```
    string fileData, Type expectedException)
```

```
{
```

```
    FileItem file = CreateValidFileItem();
```

```
    Assert.Throws(expectedException,
```

```
        () => file.Upload(fileData));
```

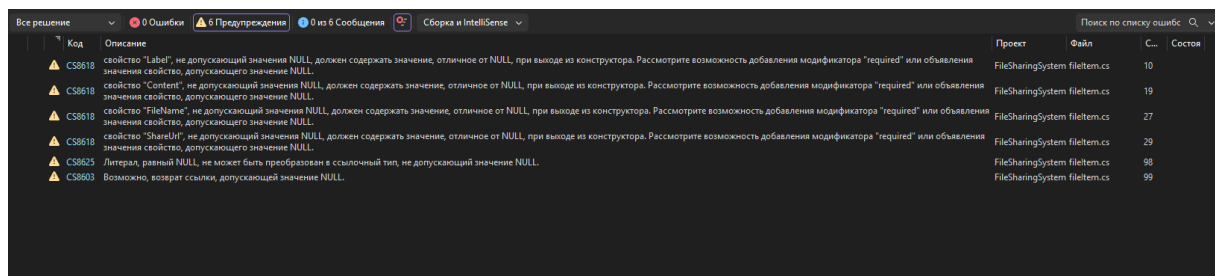
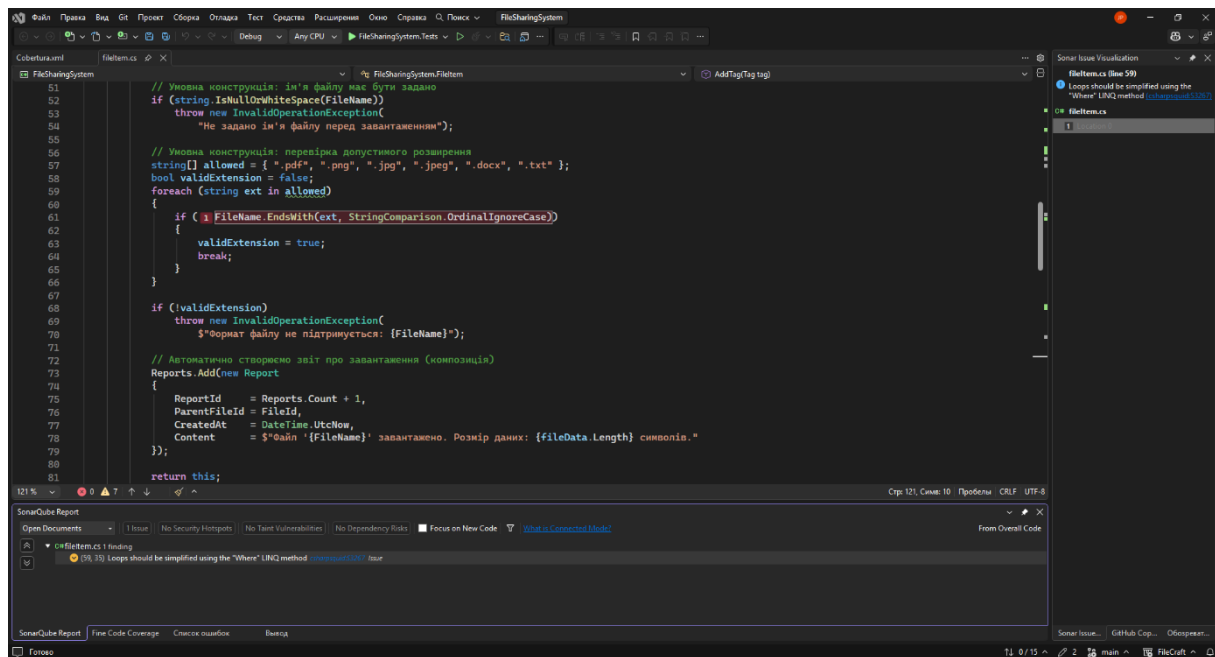
```
}
```

Чому: три тести перевіряли одну й ту саму логіку валідації з різними «порожніми» значеннями — класична Code Duplication.

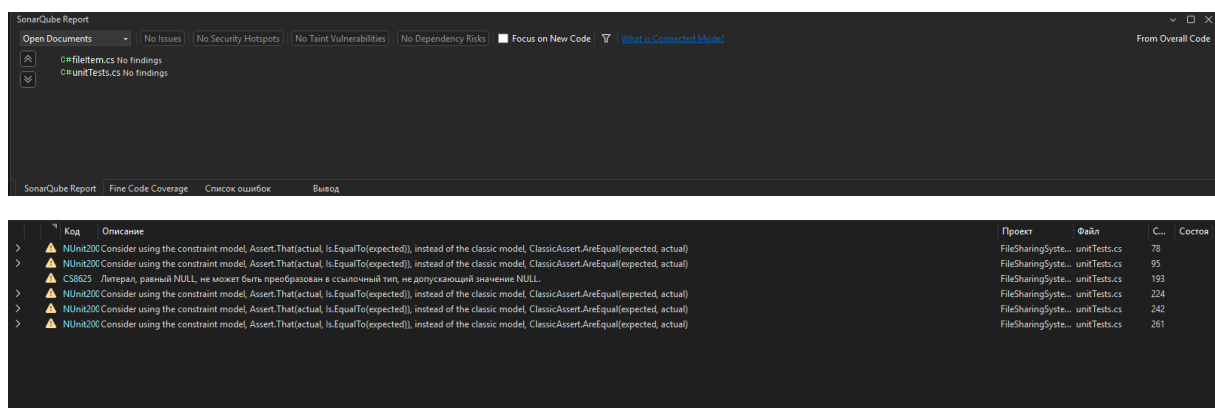
NUnit [TestCase] усуває дублювання і додає нові кейси одним рядком.

Регрессивне тестування:

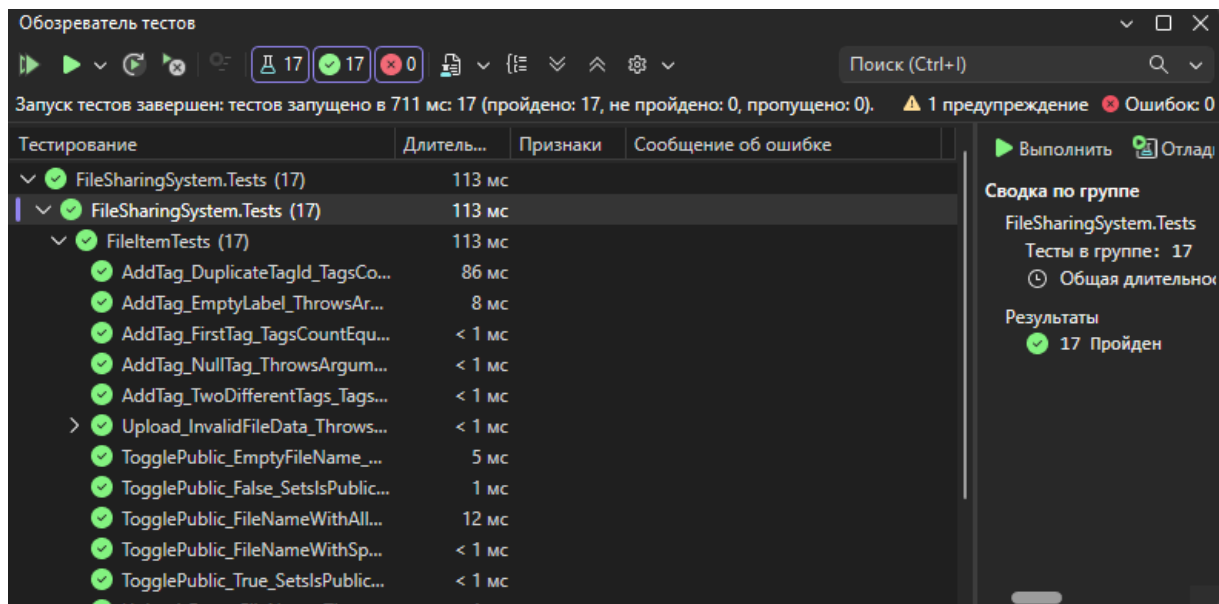
Скріншоти Sonar Cube та попереджень до:



Скріншоти Sonar Cube та попереджень після:



Результат тестів після (all green):



Метрики кода:

До:

Иерархия	Индекс удобства поддержки	Сложность организации циклов	Глубина наследования	Взаимозависимость классов	Строки исходного кода	Строки исполняемого кода
FileSharingSystem (Debug)	93	39	1	13	123	30
FileItem	79	27	1	13	104	28
Report	100	8	1	1	8	1
Tag	100	4	1	0	6	1

```
PS C:\Users\yulia\Downloads\tests\VisualCraft-main\FileSharingSystem> python -m lizard

=====
NLOC  CCN  token  PARAM  length  location
-----
33    6    164    1      44  FileItem:Upload@99-82@.FileItem.cs
23    6    119    1      30  FileItem:TogglePublic@86-115@.FileItem.cs
35    4    76     1      21  FileItem:AddTag@120-140@.FileItem.cs
4 file analyzed.
=====
NLOC  Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
3      0.0      0.0      0.0         0      .\obj\Debug\net10.0\NETCoreApp,Version=v10.0.AssemblyAttributes.cs
9      0.0      0.0      0.0         0      .\obj\Debug\net10.0\FileSharingSystem.AssemblyInfo.cs
7      0.0      0.0      0.0         0      .\obj\Debug\net10.0\FileSharingSystem.GlobalUsings.g.cs
98     23.7     5.3     119.7       3      .FileItem.cs
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
117        23.7     5.3     119.7       3         0         0.00   0.00
PS C:\Users\yulia\Downloads\tests\VisualCraft-main\FileSharingSystem>
```

Після:

```
PS C:\Users\yulia\OneDrive\Desktop\OUI\VisualCraft\FileSharingSystem> python -m lizard

=====
NLOC  CCN  token  PARAM  length  location
-----
6      1      25     3       6  FileItem:FileItem@11-36@.FileItem.cs
17     5     88     1      21  FileItem:Upload@48-68@.FileItem.cs
5      1     26     1       5  FileItem:IsValidExtension@70-74@.FileItem.cs
10     1     39     1      10  FileItem:CreateUploadReport@76-85@.FileItem.cs
18     3     72     1      22  FileItem:TogglePublic@88-109@.FileItem.cs
12     4     74     1      15  FileItem:AddTag@112-126@.FileItem.cs
4 file analyzed.
=====
NLOC  Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
3      0.0      0.0      0.0         0      .\obj\Debug\net10.0\NETCoreApp,Version=v10.0.AssemblyAttributes.cs
9      0.0      0.0      0.0         0      .\obj\Debug\net10.0\FileSharingSystem.AssemblyInfo.cs
7      0.0      0.0      0.0         0      .\obj\Debug\net10.0\FileSharingSystem.GlobalUsings.g.cs
100    11.3     2.5     54.0        6      .FileItem.cs
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
119        11.3     2.5     54.0        6         0         0.00   0.00
PS C:\Users\yulia\OneDrive\Desktop\OUI\VisualCraft\FileSharingSystem>
```

Цикломатична складність: AvgCCN: 5.3 → 2.5 (-53%)

Підсумкова рефлексія:

У ході цієї роботи продовжувалось використання Visual Studio.

Найціннішим відкриттям для мене стало те що навіть якщо код не видає помилок і є зрозумілим для автора, також важливо щоб він був зрозумілим для стороннього читача. Наприклад деякі не шкідливі деталі виявились не зрозумілими для читача що є проблемою читабельності коду. Також я зрозуміла що іноді саме сторонній читач може підказати що краще виправити у коді такого, чого не бачить сам автор. Ще я зрозуміла поняття та важливість метриків, завдяки яким можна об'єктивно оцінити складність і читабельність коду, без суб'єктивних суджень, що дає змогу справді покращити код і побачити результат у цифрах. Наприклад розбиття одного методу на три частини не збільшило загальну складність системи, але зробило кожен частину тривіально зрозумілою. Review навчило мене дивитись на код очима того хто бачить його вперше і думати про зрозумілість коду з цієї перспективи.